

TravelBudget

Project Report

AI-Powered Budget Travel Optimization Platform

Report Generated: 5/18/2026

Table of Contents

1. Project Overview
2. Architecture & Technology Stack
3. Microservices Architecture
4. Project Structure
5. Key Features
6. Technology Dependencies
7. Deployment & Infrastructure
8. API Services
9. Database Design
10. Security & Best Practices

1. Project Overview

TravelBudget is an innovative AI-powered travel optimization platform designed to help users plan budget-conscious trips with personalized itineraries and cost optimization. The application combines modern web technologies with advanced microservices architecture to provide a scalable and maintainable solution.

Project Goals:

- Provide AI-driven travel planning and itinerary generation
- Optimize travel budgets across multiple destinations
- Integrate with Indian Railways for train booking
- Enable seamless payment processing
- Deliver personalized travel recommendations

Target Users:

- Budget-conscious travelers
- Families planning group trips
- Business travelers
- Adventure seekers with flexible budgets

2. Architecture & Technology Stack

Frontend Stack:

- Next.js 14.0.4 - React framework with SSR/SSG capabilities
- React 18.2.0 - UI library
- Tailwind CSS 3.3.6 - Utility-first CSS framework
- Zustand 4.4.7 - State management
- Axios 1.6.2 - HTTP client
- Recharts 2.10.3 - Data visualization
- Framer Motion 10.16.16 - Animation library
- React Hot Toast 2.4.1 - Notifications

Backend Stack:

- Node.js 18+ - JavaScript runtime
- Express.js 4.18.2 - Web framework
- MongoDB 8.0.0 - NoSQL database
- Mongoose - ODM for MongoDB
- JWT - Authentication mechanism
- Razorpay 2.9.2 - Payment gateway

Infrastructure:

- Docker & Docker Compose - Containerization
- AWS Elastic Beanstalk - Deployment platform
- MongoDB Atlas - Cloud database
- Google OAuth - Authentication

3. Microservices Architecture

The application follows a microservices architecture pattern with an API Gateway as the central entry point. Each service is independently deployable and scalable.

Architecture Components:

API Gateway (Port 5000)

- Entry point for all client requests
- Request routing to microservices
- Health monitoring and service discovery
- CORS handling and rate limiting

Auth Service (Port 5001)

- User registration and authentication
- JWT token generation and verification
- OTP generation and verification
- Google OAuth integration
- Password reset functionality

Travel Service (Port 5002)

- Travel planning and trip management
- AI-powered itinerary generation
- Budget allocation and optimization
- Trip details and recommendation system

Train Service (Port 5003)

- Train search and availability
- Transport booking integration
- Facilities information
- Railway API integration

Payment Service (Port 5004)

- Payment initiation and verification
- Booking management
- Refund processing
- Transaction history tracking

Service Communication Diagram

Client (Next.js - Port 3000)

%

 $0\frac{1}{4}$

API Gateway (Port 5000)

% % % % % % % %<% % % % % % %, % % % % % % % % \$

$$\frac{0}{0^{1/4}} \quad \frac{0}{0^{1/4}} \quad \frac{0}{0^{1/4}} \quad \frac{0}{0^{1/4}}$$

Auth Travel Train Payment

(5001)(5002)(5003) (5004)

% % % % % % %4% % % % % % %, % % % % % % %

%

 $\frac{0}{0}^{\frac{1}{4}}$

MongoDB

(Port 27017)

4. Project Structure

Client Directory (/client)

- /app - Next.js App Router pages and layouts
- /components - Reusable React components
- /store - Zustand state management stores
- /utils - Utility functions and helpers
- Config files: next.config.js, tsconfig.json, tailwind.config.js

Server Directory (/server)

- /controllers - Request handlers for business logic
- /models - Mongoose schemas (User, Booking, Hotel, etc.)
- /routes - API route definitions
- /services - Business logic layer
- /middleware - Express middleware
- /config - Database and email configuration
- /utils - Helper functions and utilities
- /rail-api - Indian Railways API integration

Services Directory (/services)

- /auth-service - Authentication microservice
- /gateway - API Gateway
- /payment-service - Payment processing
- /train-service - Train booking service
- /travel-service - Travel planning service

5. Key Features

Authentication & Authorization

- Email/Password registration and login
- Google OAuth integration
- OTP-based password reset
- JWT-based secure token management
- Role-based access control

Travel Planning

- AI-powered itinerary generation
- Multi-destination trip planning
- Budget allocation and optimization
- Real-time availability checking
- Hotel and restaurant recommendations

Train Booking

- Integration with Indian Railways
- Real-time train search
- Facility information display
- Booking management

Payment Processing

- Razorpay payment gateway integration
- Secure transaction processing
- Refund management
- Transaction history

User Management

- Profile management
- Booking history
- Trip tracking
- Favorites and saved plans

6. Key Technology Dependencies

Frontend Dependencies (Client):

- next@14.0.4
- react@18.2.0
- react-dom@18.2.0
- tailwindcss@3.3.6
- zustand@4.4.7
- axios@1.6.2
- recharts@2.10.3
- framer-motion@10.16.16
- react-hot-toast@2.4.1
- @react-oauth/google@0.13.4

Backend Dependencies (Server):

- express@4.18.2
- mongoose@8.0.0
- jsonwebtoken@9.0.2
- bcryptjs@2.4.3
- nodemailer@6.9.7
- razorpay@2.9.2
- axios@1.6.2
- cors@2.8.5
- helmet@7.1.0
- express-rate-limit@7.1.5
- express-validator@7.0.1

7. Deployment & Infrastructure

Docker Containerization:

- Each microservice has its own Dockerfile
- Docker Compose for local development orchestration
- Service health checks configured
- Volume persistence for MongoDB data

Environment Variables:

- JWT_SECRET - Secret key for JWT signing
- MONGODB_URI - Database connection string
- SMTP_USER, SMTP_PASS - Email configuration
- RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET - Payment gateway
- ALLOWED_ORIGINS - CORS configuration

AWS Deployment:

- Elastic Beanstalk for application hosting
- .elasticbeanstalk configuration for EB deployment
- appspec.yml for CodeDeploy configuration
- Auto-scaling and load balancing support

Database:

- MongoDB with authentication enabled
- Collections: Users, Bookings, Hotels, Restaurants, Trips
- Connection pooling for performance
- Automated backup and recovery

8. API Services Overview

Auth Service Endpoints:

POST /auth/register - User registration
POST /auth/login - User login
POST /auth/verify-otp - OTP verification
POST /auth/reset-password - Password reset
POST /auth/google-oauth - Google OAuth login

Travel Service Endpoints:

POST /travel/generate-plan - AI itinerary generation
GET /travel/trips - Fetch user trips
POST /travel/trips - Create new trip
GET /travel/recommendations - Get recommendations
POST /travel/optimize-budget - Budget optimization

Train Service Endpoints:

POST /trains/search - Search trains
GET /trains/:id - Get train details
POST /trains/:id/book - Book train ticket
GET /trains/facilities - Get facility information

Payment Service Endpoints:

POST /payments/initiate - Initiate payment
POST /payments/verify - Verify payment
POST /payments/refund - Process refund
GET /payments/history - Transaction history

9. Database Design

User Model:

- _id, email, password, firstName, lastName
- googleId, googleAccessToken
- phone, address, city, state, country
- createdAt, updatedAt

Trip Model:

- _id, userId, title, description
- startDate, endDate, status
- budget, spent, destinations
- itinerary, bookings, preferences

Booking Model:

- _id, userId, tripId, type (hotel/train/restaurant)
- bookingDetails, confirmationNumber
- amount, paymentStatus, paymentId
- createdAt, cancelledAt

Hotel & Restaurant Models:

- _id, name, location, rating
- price, amenities, images
- reviews, availability

Transport Model:

- _id, type, name, departure, arrival
- price, availableSeats, facilities
- schedule, cancellationPolicy

10. Security & Best Practices

Authentication Security:

- JWT tokens with 7-day expiration
- Password hashing with bcryptjs
- Secure OTP generation and verification
- OAuth 2.0 integration for third-party auth

API Security:

- CORS restrictions with whitelist
- Rate limiting on sensitive endpoints
- Request validation with express-validator
- Helmet.js for security headers

Data Protection:

- HTTPS enforcement in production
- Encrypted password storage
- Sensitive data encryption in transit
- MongoDB authentication enabled

Code Quality:

- Error handling middleware
- Request logging with Morgan
- Input validation on all endpoints
- Environment variable management

Infrastructure Security:

- Docker container isolation
- Network segmentation with Docker networks
- Health checks for service monitoring
- Automated rollback on deployment

Development Workflow

Local Development Setup:

1. Clone the repository
2. Install dependencies: `npm install`
3. Configure environment variables
4. Run Docker Compose: `docker-compose up`
5. Start Next.js dev server: `npm run dev`

Running Tests:

- Unit tests: `npm test`
- Integration tests: `npm run test:integration`
- HTTP endpoint tests: `node test-http.js`

Building for Production:

- Build frontend: `npm run build`
- Docker image build: `docker build -t image-name .`
- Deploy to AWS Elastic Beanstalk: `eb deploy`

Conclusion

TravelBudget represents a modern, scalable approach to travel planning with AI-powered personalization. The microservices architecture ensures flexibility and independent scalability of services, while the technology stack provides robustness and developer familiarity.

The project demonstrates best practices in:

- Microservices architecture patterns
- API design and documentation
- Security implementation
- Containerization and deployment
- State management and UI/UX design

Continued development should focus on:

- Expanding AI capabilities for recommendations
- Adding more travel provider integrations
- Implementing real-time notifications
- Enhanced mobile app support
- Performance optimization and caching

Report Generated: Monday, May 18, 2026